**UNIVERSIDAD AUTÓNOMA DE OCCIDENTE**
**Facultad de Ingeniería y Ciencias Básicas**

# Recomendador de seguros para personas basado en embeddings multilingües y ranking supervisado: propuesta formal para Seguros Bolívar

Maestría en Inteligencia Artificial y Ciencia de Datos
Asignatura: Gestión de Proyectos en IA y CD
Módulo 2 · Propuesta del proyecto

**Grupo 5**
Daniel Alejandro Silva Ramírez
Ana Katherine Medina Zuleta (Code Reviewer)

Docente: Jan Polanco Velasco
Fecha de radicación: 31 de octubre de 2026
Santiago de Cali

## Tabla de contenido

1. Contexto, modelo base y restricciones 3

2. Objetivos y matriz de alcance 5

3. Cronograma y ruta crítica (CPM) 6

4. Marco de investigación y CRISP-DM 10

5. Historias de usuario, criterios de aceptación y Kanban 13

6. Anexos y enlaces 18

# 1. Contexto, modelo base y restricciones

## 1.1 Problema y usuarios (Data Thinking)

Una aseguradora de personas vende más de veinte líneas de producto por canales digitales, asesores y la red bancaria aliada. Hoy la oferta de un segundo o tercer seguro depende de campañas masivas y de la intuición del asesor: el cliente que tiene carro recibe la misma campaña que quien no lo tiene, y los productos de nicho (mascotas, arrendamiento, salud internacional) casi no se ofrecen. El resultado es una venta cruzada baja: en la cartera de referencia cada cliente tiene en promedio 2,47 pólizas y el 24,3 % tiene una sola.

Siguiendo el enfoque de Data Thinking se partió de la pregunta: *¿Cómo podríamos ayudar a los asesores a ofrecer a cada cliente el siguiente seguro que realmente necesita, sin depender de campañas masivas ni de la intuición?* Los actores son el **cliente persona** (recibe ofertas irrelevantes), el **asesor** (no sabe qué ofrecer y a quién), el **gerente de producto** (necesita aumentar pólizas por cliente), el **equipo de datos** (debe operar el modelo con trazabilidad) y **cumplimiento** (protección de datos y trato justo). Los datos disponibles son el perfil del cliente, su historial de pólizas y siniestros y el catálogo de productos.

## 1.2 Modelo de negocio (Business Model Canvas)

<table>
<tr>
<td rowspan="2">Socios clave<br>Red bancaria aliada; equipo de datos y gobierno de datos; proveedor del modelo abierto (Hugging Face).</td>
<td>Actividades clave<br>Mantener el contrato de datos, reentrenar el ranking, auditar sesgo y monitorear métricas.</td>
<td rowspan="2">Propuesta de valor<br>Top-5 de seguros personalizado y explicado para cada cliente, también para clientes nuevos, sin enviar datos fuera de la compañía.</td>
<td>Relación con clientes<br>Asistida: el asesor decide y explica; autoservicio en el canal digital.</td>
<td rowspan="2">Segmentos<br>Clientes personas con al menos una póliza; asesores y canal digital como usuarios internos.</td>
</tr>
<tr>
<td>Recursos clave<br>Historial de pólizas anonimizado, catálogo, modelo e5-small, infraestructura CPU y MLflow.</td>
<td>Canales<br>Consola de asesores (UI), API para app y web, campañas segmentadas.</td>
</tr>
<tr>
<td colspan="2">Estructura de costos<br>Horas de ingeniería (dos personas, 20 SP por sprint), cómputo CPU (US$ 0 en local y en la capa gratuita de Cloud Run, sección 3.3), gobierno y auditoría de datos. Sin licencias: modelo MIT y librerías abiertas.</td>
<td colspan="3">Fuentes de ingreso / valor<br>Más pólizas por cliente, mayor conversión de ofertas, menor costo de campañas masivas y menor cancelación por productos mal ofrecidos.</td>
</tr>
</table>

### 1.3 Ficha del modelo base (MaaS)

| Campo | Valor |
|---|---|
| Identificación | `intfloat/multilingual-e5-small` · huggingface.co/intfloat/multilingual-e5-small |
| Autoría | Microsoft (L. Wang et al.), publicado como modelo abierto en Hugging Face [1] |
| Función matemática | Codificador de texto (transformer de 12 capas, 118 M parámetros) que produce *embeddings* de 384 dimensiones para recuperación semántica por similitud coseno; entrada de hasta 512 tokens en 94 idiomas |
| Uso en el proyecto | Codificar el catálogo (`passage: …`) y el perfil del cliente (`query: …`) para generar candidatos; resuelve el arranque en frío |
| Licencia | MIT: permite uso comercial, modificación y redistribución |

### 1.4 Resumen de la Model Card y análisis ético [4]

**Entrenamiento.** Preentrenamiento contrastivo débilmente supervisado sobre pares de texto multilingües a gran escala y ajuste fino con conjuntos etiquetados de recuperación [1]. **Métricas reportadas.** La Model Card publica resultados de recuperación multilingüe y de la suite MTEB; el proyecto no los toma como garantía y mide su propio desempeño con Recall@5 y NDCG@5. **Limitaciones.** Trunca textos de más de 512 tokens; exige los prefijos `query:` y `passage:`; su calidad baja en registros coloquiales poco representados en la web. **Sesgos.** Al aprender de textos de internet puede codificar estereotipos de género o región. Mitigación: el perfil textual excluye género, nombre e identificadores (HU-04) y el desempeño se audita por estrato, ciudad y género (HU-09). **Decisión humana.** La recomendación es un apoyo; el asesor decide y el cliente puede rechazarla.

### 1.5 Matriz de restricciones técnicas

| Dimensión | Restricción |
|---|---|
| Software | Python 3.13 con `uv`; PyTorch CPU y sentence-transformers; LightGBM; FastAPI; Streamlit; MLflow; pytest y ruff |
| Hardware | CPU de 4 núcleos, 8–16 GB de RAM; sin GPU. Pesos del modelo ≈ 470 MB; imagen Docker objetivo ≤ 2,5 GB |
| Datos | Sintéticos para desarrollo; reales solo anonimizados, con autorización escrita de la gerencia y dentro de la infraestructura de la compañía |
| Licencias | Modelo MIT; LightGBM MIT; FastAPI MIT; Streamlit Apache 2.0. Se descartó TabPFN porque sus versiones recientes tienen licencia no comercial [9] |

| Dimensión | Restricción |
|---|---|
| Ejecución | Inferencia local y sin internet: ningún dato de cliente se envía a APIs externas |

## 2. Objetivos y matriz de alcance

### 2.1 Objetivo general

Construir un servicio desacoplado que recomiende a cada cliente persona los cinco seguros más relevantes que aún no tiene, combinando *embeddings* de `multilingual-e5-small` con un ranking LightGBM, expuesto mediante API FastAPI e interfaz para asesores y empaquetado en Docker CPU, listo para operar con datos reales anonimizados.

### 2.2 Objetivos específicos

1. Generar candidatos por similitud semántica con latencia p95 < 50 ms por cliente en CPU.
2. Superar la línea base de segmento en evaluación *leave-last-out*: Recall@5 ≥ 0,70 y NDCG@5 ≥ 0,48 (base: 0,639 y 0,435).
3. Exponer una API con p95 < 200 ms y validación que rechace datos personales.
4. Mantener una brecha de Recall@5 ≤ 10 puntos porcentuales entre estratos, ciudades y géneros.
5. Entregar una imagen Docker reproducible gobernada por CI con cobertura de pruebas ≥ 85 %.

### 2.3 OKR y KPI

| Objetivo (O) | Resultados clave (KR) |
|---|---|
| **O1.** Que cada cliente reciba ofertas de seguros relevantes | KR1 Recall@5 ≥ 0,70 · KR2 cobertura del catálogo ≥ 80 % · KR3 brecha entre grupos ≤ 10 pp |
| **O2.** Dejar un servicio confiable listo para datos reales | KR1 0 columnas PII aceptadas por el contrato · KR2 imagen Docker que corre sin internet · KR3 CI en verde con cobertura ≥ 85 % |

| KPI | Tipo | Línea base | Meta | Medición |
|---|---|---|---|---|
| Recall@5 | Técnico | 0,639 | ≥ 0,70 | Evaluación offline por versión (MLflow) |

| KPI | Tipo | Línea base | Meta | Medición |
|---|---|---|---|---|
| NDCG@5 | Técnico | 0,435 | ≥ 0,48 | Evaluación offline por versión |
| Latencia p95 de la API | Técnico | — | < 200 ms | Prueba de carga en CPU |
| Cobertura del catálogo | Técnico | 53 % | ≥ 80 % | Productos distintos en los top-5 |
| Pólizas por cliente | Negocio | 2,47 | +5 % en piloto | Cartera, 6 meses tras el piloto |
| Conversión de ofertas | Negocio | Por medir | Superior a campaña masiva | Prueba A/B en el piloto |

### 2.4 Matriz de alcance

| Categoría | Elementos |
|---|---|
| **In** (incluido) | Generador sintético y contrato de datos; modelo base e5-small; candidatos por similitud; ranking LightGBM; MLflow; API FastAPI con la razón de cada recomendación y medición de latencia; UI para asesores; Docker CPU; CI; auditoría de sesgo |
| **Nice to have** | Exportación ONNX para menor latencia; monitoreo de deriva de datos; especificación SDD y AGENTS.md; comparación académica con TabPFN; despliegue en Hugging Face Spaces o en la nube corporativa |
| **Out** (excluido) | Tarificación, suscripción o decisiones de elegibilidad; envío automático de ofertas sin asesor; entrenamiento del modelo base desde cero; GPU dedicada; datos reales sin anonimizar |

## 3. Cronograma y ruta crítica (CPM)

El sprint de construcción (M3) dura 15 días calendario con dos integrantes. Cada historia es una tarea atómica (1 SP = 1 día, regla 1:1 del curso). La preparación (entorno `uv`, datos sintéticos, contrato y líneas base: HU-01 y HU-02) ya se completó durante el M2 y es la base del sprint. El cronograma y la tabla CPM se calculan automáticamente desde `docs/backlog.json`, la misma fuente del Capítulo 5 y del tablero.

## 3.1 Diagrama de Gantt

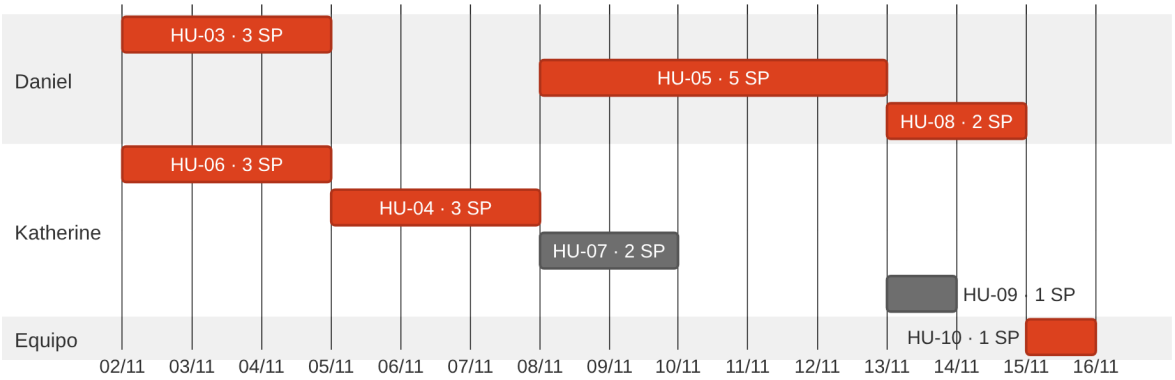



Figura 1. Gantt del sprint M3 (2 al 16 de noviembre de 2026), una fila por integrante. Las barras rojas forman la ruta crítica.

## 3.2 Ruta crítica (CPM)

**Tabla 1. Cálculo CPM (días; ES/EF = inicio/fin temprano, LS/LF = inicio/fin tardío). En rojo, holgura cero**

| Tarea | Depende de | Dur. | ES | EF | LS | LF | Holgura |
|---|---|---|---|---|---|---|---|
| HU-03 Carga segura del modelo base e5-small | — | 3 | 0 | 3 | 0 | 3 | 0 |
| HU-04 Perfil textual y candidatos por similitud | HU-03, HU-06* | 3 | 3 | 6 | 3 | 6 | 0 |
| HU-05 Ranking LightGBM con trazabilidad en MLflow | HU-04 | 5 | 6 | 11 | 6 | 11 | 0 |
| HU-06 API REST de recomendaciones | — | 3 | 0 | 3 | 0 | 3 | 0 |
| HU-07 Interfaz para asesores | HU-06, HU-04* | 2 | 6 | 8 | 10 | 12 | 4 |
| HU-08 Docker CPU e integración continua | HU-05, HU-06 | 2 | 11 | 13 | 11 | 13 | 0 |
| HU-09 Auditoría de sesgo por segmento | HU-05, HU-07* | 1 | 11 | 12 | 12 | 13 | 1 |
| HU-10 Integración, pruebas de humo y entrega | HU-08, HU-09 | 1 | 13 | 14 | 13 | 14 | 0 |

* Dependencia de recurso: la tarea la realiza la misma persona y no puede ejecutarse en paralelo.

**Ruta crítica:** HU-03 → HU-04 → HU-05 → HU-08 → HU-10 = 14 días, holgura cero. Incluye las tres tareas críticas del curso: **descarga y verificación de pesos** (HU-03), **contratos numéricos del preprocesamiento** (perfil y vectores, HU-04) y las

**dependencias C++ en Docker** (LightGBM requiere `libgomp` en la imagen, HU-08). La API (HU-06) arranca el día 0 en paralelo con un recomendador simulado; también queda con holgura cero porque Katherine debe terminarla antes de empezar HU-04 (dependencia de recurso). La ruta deja 1 día(s) de colchón frente a los 15 del sprint. Los días 3 a 6 Daniel no tiene tarea crítica asignada y apoya la revisión de PR y la preparación de variables para HU-05.

**Tabla 2. Carga por integrante (capacidad individual = 15 días × 75 % = 11,25 SP)**

| Integrante | Historias | SP | Capacidad | Uso |
|---|---|---|---|---|
| Daniel Silva | HU-03, HU-05, HU-08, HU-10 | 10,50 | 11,25 | 93 % |
| Katherine Medina | HU-04, HU-06, HU-07, HU-09, HU-10 | 9,50 | 11,25 | 84 % |

HU-10 (1 SP) se reparte entre los dos integrantes (0,5 SP cada uno).

### 3.3 Presupuesto de inferencia

El servicio no usa GPU ni APIs externas: cada petición codifica un perfil corto (≈ 60 tokens) con e5-small en CPU, compara contra 19 vectores de producto precalculados y ordena hasta 17 candidatos con LightGBM. Las cifras de latencia son **estimaciones de diseño** para un núcleo de CPU moderno; HU-06 mide el p95 de la API y HU-08 verifica la imagen limitada a 1 vCPU y 2 GiB. Si la medición supera el presupuesto, la primera palanca es exportar a ONNX (Nice to have) y la segunda reducir a INT8.

**Tabla 3. Presupuesto por petición (CPU, 1 vCPU)**

| Etapa | Estimado p95 | Presupuesto | Medición |
|---|---|---|---|
| Validación Pydantic y serialización FastAPI | ≈ 5 ms | 20 ms | HU-06 |
| Codificación del perfil con e5-small (PyTorch CPU) | ≈ 30 ms | 100 ms | HU-03 |
| Similitud coseno contra 19 vectores precalculados | < 1 ms | 10 ms | HU-04 (p95 < 50 ms incl. codificación) |
| Ranking LightGBM de ≤ 17 candidatos | ≈ 2 ms | 30 ms | HU-05 |
| Razón legible de cada producto del top-5 | ≈ 5 ms | 40 ms | HU-06 |
| **Total de la petición** | **≈ 45 ms** | **< 200 ms** | HU-06 (1.000 peticiones) y HU-08 |

**Recursos.** Pesos fp32 de ≈ 470 MB; memoria residente esperada ≈ 1 GiB por proceso (meta < 1,5 GiB); contenedor de 1 vCPU y 2 GiB; imagen ≤ 2,5 GB. El arranque en frío (carga del modelo) se estima en 5–10 s y no cuenta dentro del p95 de la API.

**Tabla 4. Costo mensual estimado de inferencia (precios Tier 1 de Google Cloud Run consultados el 29/10/2026 [11])**

| Escenario | Peticiones/mes | vCPU-s | GiB-s | Costo (USD) |
|---|---|---|---|---|
| Local o servidor propio de la compañía | ilimitado | — | — | ≈ 0 (hardware existente) |
| Piloto: asesores de una regional | 100.000 | 10.000 | 20.000 | 0 (capa gratuita) |
| Operación nacional: asesores + canal digital | 1.000.000 | 100.000 | 200.000 | 0 (capa gratuita) |
| Campaña masiva | 5.000.000 | 500.000 | 1.000.000 | ≈ 10,48 |
| Instancia siempre activa (sin arranque en frío) | cualquiera | 2.592.000 | 5.184.000 | ≈ 55,5 |

Supuestos: facturación por petición con redondeo a 100 ms (0,1 vCPU-s y 0,2 GiB-s por petición); CPU US$ 0,000024 por vCPU-s, memoria US$ 0,0000025 por GiB-s, US$ 0,40 por millón de peticiones; capa gratuita mensual de 180.000 vCPU-s, 360.000 GiB-s y 2 millones de peticiones. Campaña: (500.000 − 180.000) × 0,000024 + (1.000.000 − 360.000) × 0,0000025 + 3 × 0,40 = 7,68 + 1,60 + 1,20. Instancia siempre activa: tarifa por instancia de US$ 0,000018 por vCPU-s durante 30 días, descontada la capa gratuita.

**Comparación con un LLM vía API.** Resolver la misma recomendación enviando el perfil y el catálogo a un modelo de lenguaje comercial (≈ 1.500 tokens de entrada y 150 de salida con Claude Haiku 5.5: US$ 0,10 y US$ 0,50 por millón de tokens) costaría ≈ US$ 0,000225 por petición, es decir ≈ US$ 225 al mes en el escenario nacional, con latencias de segundos y, sobre todo, enviando datos de clientes a un tercero, lo que viola la restricción de ejecución local de la sección 1.5. El modelo abierto en CPU es más barato, más rápido y cumple la Ley 1581.

# 4. Marco de investigación y CRISP-DM

## 4.1 Tecnologías, formatos y datos

| Elemento | Opciones evaluadas | Decisión |
|---|---|---|
| Formato de pesos | `.safetensors` (carga sin ejecutar código) · `.bin`/`.pt` (pickle: puede ejecutar código arbitrario) · `.onnx` (interoperable, inferencia rápida) | `.safetensors` con revisión fijada; ONNX como Nice to have |
| Modelo de ranking | Popularidad, segmento, LightGBM LambdaRank [3][8], TabPFN [9] | LightGBM, guardado en texto (sin pickle) |
| Datos | Sintéticos propios · públicos (Insurance Company Benchmark COIL 2000, UCI) · reales anonimizados | Sintéticos con el mismo contrato que los reales |
| Técnicas | *Embeddings* densos [1][2], filtrado colaborativo, ranking supervisado | Híbrido: candidatos semánticos + ranking |

**Dataset sintético.** El generador produce, con semilla 42, 20.000 clientes, 19 productos, 49.498 pólizas y 39.915 siniestros entre 2014-01-05 y 2026-09-30. No incluye nombres, documentos ni contactos; los nulos aparecen solo en `fecha_fin` (pólizas vigentes, 89,1 %). La compra se simula con propensión por perfil (solo quien tiene carro compra Autos) y venta cruzada (Autos → Accidentes, Hogar → Mascotas, Salud → Renta por hospitalización), para que exista una señal aprendible. Los productos más frecuentes son Seguro de Vida Individual (21,8 %), Seguro de Salud Integral (18,8 %), Seguro de Hogar Digital (18,5 %).

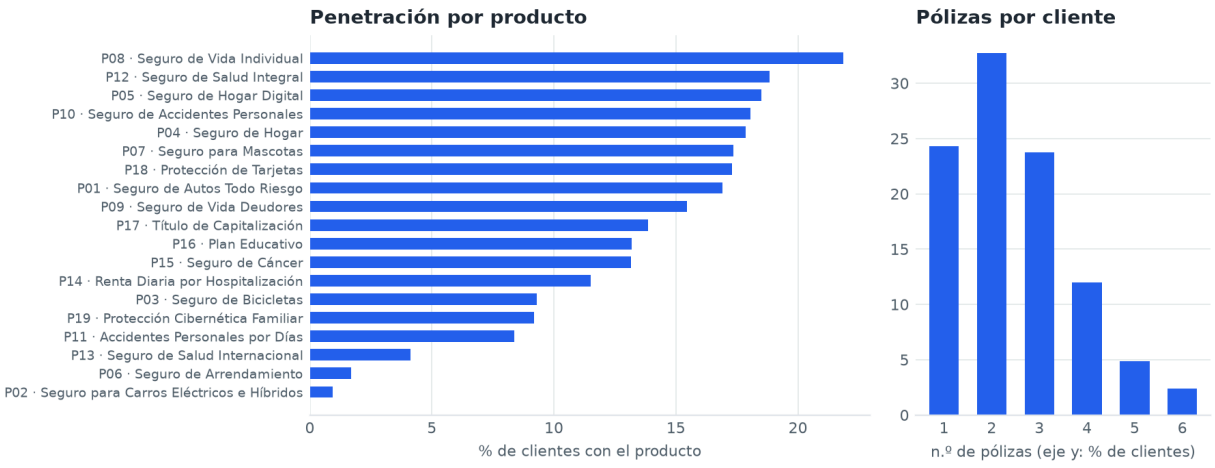



Figura 2. Penetración por producto y distribución de pólizas por cliente en el dataset sintético.

**Línea base.** Con partición *leave-last-out* (15.140 clientes con dos o más pólizas), recomendar lo más popular logra Recall@5 = 0,426 y NDCG@5 = 0,260; la popularidad por segmento (vehículo, vivienda, mascota y crédito) sube a 0,639 y 0,435. La diferencia confirma que personalizar paga y fija el umbral del modelo propuesto.

### 4.2 Arquitectura propuesta

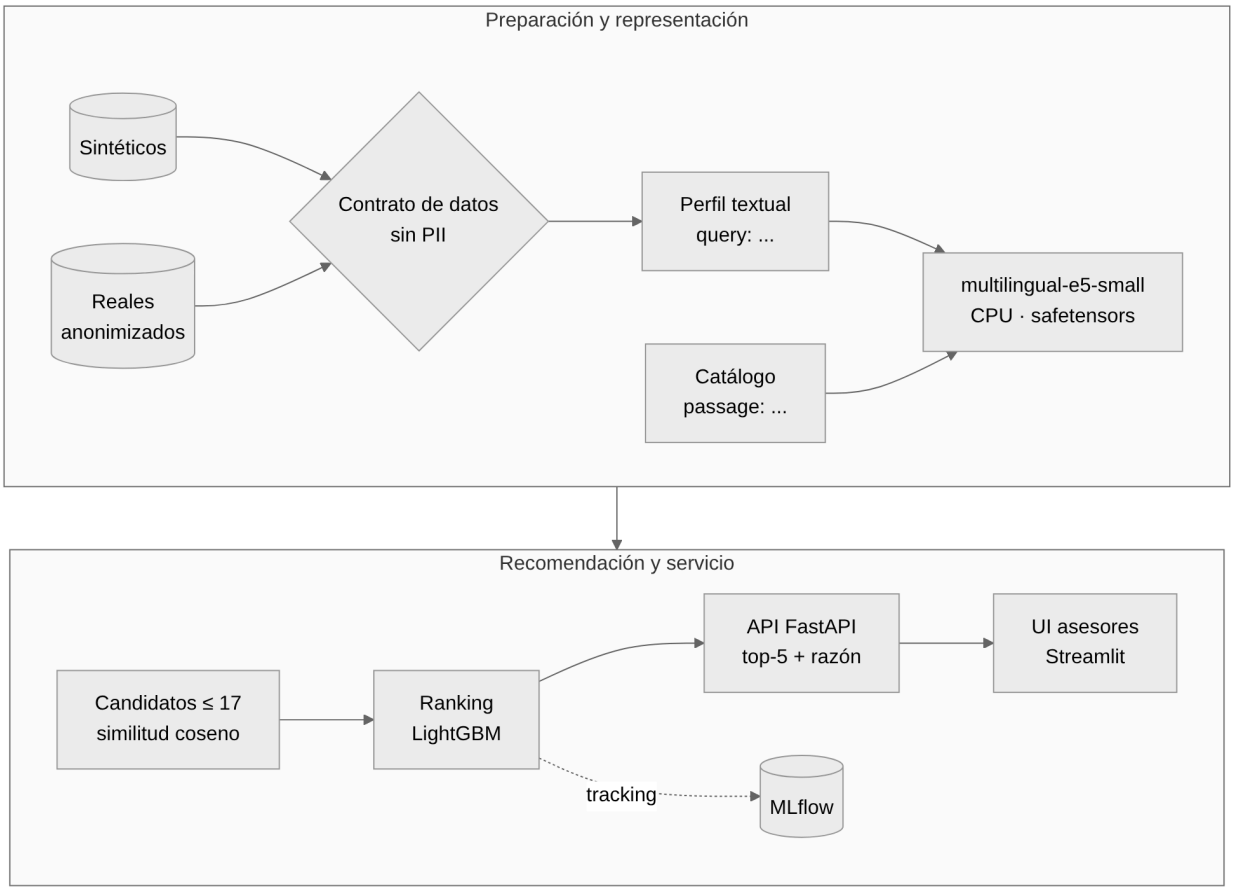



Figura 3. Flujo de inferencia: contrato de datos → perfil textual → e5-small → candidatos → ranking → API (top-5 + razón) y UI, con trazabilidad en MLflow.

### 4.3 Mapeo a las seis fases de CRISP-DM [10]

| Fase | Entregable en este proyecto |
|---|---|
| 1. Business Understanding | Pregunta HMW, Canvas, OKR y KPI (secciones 1.1, 1.2 y 2.3) |
| 2. Data Understanding | EDA, reporte de nulos y penetración por producto (`scripts/eda.py`, Figura 2) |
| 3. Data Preparation | Contrato de datos, perfil textual sin PII, partición *leave-last-out* (HU-01, HU-02, HU-04) |

| Fase | Entregable en este proyecto |
|---|---|
| 4. Modeling | Inferencia con el modelo MaaS e5-small, ranking LightGBM y trazabilidad en MLflow (HU-03, HU-05) |
| 5. Evaluation | Recall@5, NDCG@5, cobertura de catálogo, auditoría de sesgo por grupo y latencia (HU-05, HU-06, HU-09) |
| 6. Deployment | API FastAPI, UI y Docker CPU con CI (HU-06, HU-07, HU-08, HU-10) |

### 4.4 Riesgos, regulación y gobernanza

**Clasificación regulatoria.** El EU AI Act [5] considera de alto riesgo los sistemas que evalúan riesgo o fijan precios de seguros de vida y salud para personas naturales. Este proyecto **recomienda productos** y deja fuera de alcance la tarificación, la suscripción y la elegibilidad (sección 2.4), por lo que se ubica en riesgo limitado, con obligación de transparencia: la UI informa que la sugerencia proviene de un sistema de IA. En Colombia aplican la Ley 1581 de 2012 de protección de datos personales [7] y la Ley 1328 de 2009 de protección al consumidor financiero. Como referencia de gestión se declara la norma **ISO/IEC 42001:2023** [6]: roles definidos, trazabilidad de cada versión en MLflow, gestión de riesgos y mejora continua.

| Riesgo | P | I | Mitigación |
|---|---|---|---|
| Exposición de datos reales de clientes | Baja | Alta | Anonimización previa, contrato que rechaza PII, ejecución local y autorización escrita |
| Los sintéticos no representan la realidad | Alta | Media | Mismo contrato para ambos; reentrenar y reportar métricas reales por separado |
| Sesgo por estrato, ciudad o género | Media | Alta | Perfil sin género; auditoría HU-09 con brecha ≤ 10 pp |
| Uso para tarificar (reclasificación a alto riesgo) | Baja | Alta | Alcance Out explícito; el asesor siempre decide |
| Sesgo de popularidad (siempre los mismos productos) | Media | Media | KPI de cobertura del catálogo ≥ 80 % |
| Peso de la imagen Docker | Media | Media | PyTorch CPU, build multi-stage, pesos descargados por script (no en Git) |
| Equipo de dos personas | Media | Media | WIP 1 por persona, ruta crítica vigilada, PR revisado por la Code Reviewer, CI obligatorio y alcance Nice to have recortable |

| Riesgo | P | I | Mitigación |
|---|---|---|---|
| Latencia o costo de inferencia mayores a lo estimado | Baja | Media | Medición del p95 en HU-06 antes del despliegue; ONNX e INT8 como palancas |

## 5. Historias de usuario, criterios de aceptación y Kanban

**Capacidad.** 2 integrantes × 15 días × 75 % de foco = **22,5 SP**. El sprint planifica **8 historias y 20 SP** (89 % de la capacidad): el margen restante absorbe revisiones de PR, reuniones y la incertidumbre de HU-05. HU-01 y HU-02 (4 SP) se completaron en el M2 y figuran en Done. La carga por persona está en la Tabla 2.

**Estimación Fibonacci 1:1.** Se estimó por comparación con una historia de referencia (HU-01, 2 SP: tarea conocida, sin incertidumbre técnica, un día y medio a dos de trabajo). Cada historia se ubica en 1, 2, 3, 5, 8 o 13 según tres factores: *esfuerzo* (cantidad de trabajo), *complejidad* (dificultad técnica) e *incertidumbre* (lo que aún no se sabe). Ninguna historia supera 5 SP; una de 8 o más se habría dividido. La justificación de cada número aparece al pie de cada tarjeta.

**Criterios de aceptación.** Cada historia tiene tres criterios verificables en formato **Given-When-Then** (*Dado* un contexto, *cuando* ocurre una acción, *entonces* se observa un resultado medible), que se traducen directamente en pruebas de pytest con patrón AAA.

**Tablero.** GitHub Projects con las columnas To Do, In Progress (máximo una tarea por integrante), Review / PR (aprobación de la Code Reviewer) y Done (fusionado en `develop` con pruebas en verde). Los issues se generan y actualizan con `scripts/crear_kanban.py` desde `docs/backlog.json`, la misma fuente de este capítulo, con las etiquetas ML, Backend, Frontend y Testing; así título, puntos, responsable y criterios del tablero coinciden con el documento.

**Tabla 5. Backlog del proyecto**

| ID | Historia | SP | Etiqueta | Responsable | Columna |
|---|---|---|---|---|---|
| HU-01 | Dataset sintético con contrato de datos | 2 | Backend | Katherine Medina | Done |
| HU-02 | Líneas base y partición temporal | 2 | ML | Daniel Silva | Done |
| HU-03 | Carga segura del modelo base e5-small | 3 | ML | Daniel Silva | To Do |

| ID | Historia | SP | Etiqueta | Responsable | Columna |
|---|---|---|---|---|---|
| HU-04 | Perfil textual y candidatos por similitud | 3 | ML | Katherine Medina | To Do |
| HU-05 | Ranking LightGBM con trazabilidad en MLflow | 5 | ML | Daniel Silva | To Do |
| HU-06 | API REST de recomendaciones | 3 | Backend | Katherine Medina | To Do |
| HU-07 | Interfaz para asesores | 2 | Frontend | Katherine Medina | To Do |
| HU-08 | Docker CPU e integración continua | 2 | Testing | Daniel Silva | To Do |
| HU-09 | Auditoría de sesgo por segmento | 1 | Testing | Katherine Medina | To Do |
| HU-10 | Integración, pruebas de humo y entrega | 1 | Testing | Equipo (Daniel y Katherine) | To Do |

**HU-01 · Dataset sintético con contrato de datos** — 2 SP · Backend · Katherine Medina

**Como** analista de datos, **quiero** una cartera sintética reproducible con el mismo esquema que tendrán los datos reales, **para** desarrollar y probar el recomendador sin exponer información de clientes.

1. **Dado:** la semilla 42 y 20.000 clientes. **Cuando:** se ejecuta «recomendador generar» dos veces. **Entonces:** las cuatro tablas (clientes, productos, pólizas, siniestros) son idénticas byte a byte.
2. **Dado:** una tabla con una columna «documento», un tipo incorrecto o un nulo no permitido. **Cuando:** se valida contra el contrato. **Entonces:** el contrato devuelve el error y la tabla se rechaza.
3. **Dado:** el código en la rama de la historia. **Cuando:** se ejecuta pytest. **Entonces:** tests/test_sintetico.py y tests/test_contrato.py pasan en verde.

**Estimación 2 SP:** Tarea conocida y acotada (generador con reglas de negocio + validador de esquema), sin incertidumbre técnica; equivale a la historia de referencia del equipo.

**HU-02 · Líneas base y partición temporal** — 2 SP · ML · Daniel Silva

**Como** científico de datos, **quiero** una partición leave-last-out y dos líneas base (popularidad y segmento), **para** tener un umbral objetivo que el modelo propuesto deba superar.

1. **Dado:** un cliente con dos o más pólizas. **Cuando:** se aplica la partición leave-last-out. **Entonces:** la póliza retenida es la más reciente y ninguna fecha de entrenamiento es posterior a ella.
2. **Dado:** el dataset sintético generado. **Cuando:** se ejecuta «recomendador baseline». **Entonces:** la CLI imprime Recall@5 y NDCG@5 de popularidad y de segmento.
3. **Dado:** rankings construidos a mano con resultado conocido. **Cuando:** se calculan las métricas. **Entonces:** tests/test_evaluacion.py confirma los valores esperados.

**Estimación 2 SP:** Dos algoritmos simples y dos métricas estándar con fórmula cerrada; el esfuerzo está en las pruebas de no fuga temporal. Mismo tamaño que HU-01.

**HU-03 · Carga segura del modelo base e5-small** — 3 SP · ML · Daniel Silva

**Como** ingeniero de ML, **quiero** descargar multilingual-e5-small en formato safetensors con revisión fijada y cargarlo en CPU, **para** generar embeddings reproducibles sin depender de internet en ejecución.

1. **Dado:** la revisión del modelo fijada en config.py. **Cuando:** se ejecuta el script de descarga. **Entonces:** se obtienen solo archivos .safetensors y su hash SHA-256 coincide con el registrado.
2. **Dado:** una lista de n textos con prefijo «passage: ». **Cuando:** se llama a encode(). **Entonces:** devuelve un arreglo float32 de forma (n, 384) con norma L2 = 1 ± 1e-5 y sin NaN.
3. **Dado:** HF_HUB_OFFLINE=1 y los pesos en caché. **Cuando:** se carga el modelo. **Entonces:** la carga termina sin ninguna conexión de red (tests/test_encoder.py con doble del modelo).

**Estimación 3 SP:** Integración con una librería externa conocida, pero con incertidumbre en la verificación de hash, el modo sin red y el doble de prueba del modelo; algo más que la referencia de 2 SP.

**HU-04 · Perfil textual y candidatos por similitud** — 3 SP · ML · Katherine Medina

**Como** asesor comercial, **quiero** que el sistema proponga los productos más afines al perfil del cliente, **para** atender también a clientes nuevos sin historial de compra.

1. **Dado:** un perfil de cliente. **Cuando:** se construye el texto «query: …». **Entonces:** el resultado es siempre el mismo para el mismo perfil y no contiene género, nombre ni identificadores.
2. **Dado:** un cliente que ya tiene Autos y Hogar. **Cuando:** se piden los candidatos. **Entonces:** la lista excluye esos dos productos y tiene a lo sumo 17 elementos ordenados por similitud.
3. **Dado:** los vectores del catálogo precalculados y 1.000 perfiles. **Cuando:** se generan candidatos en CPU. **Entonces:** el percentil 95 de la latencia es menor de 50 ms por cliente (tests/test_candidatos.py).

**Estimación 3 SP:** Plantilla de texto determinista y búsqueda coseno sobre 19 vectores: lógica sencilla, pero exige contratos numéricos y una meta de latencia; esfuerzo equivalente a HU-03.

**HU-05 · Ranking LightGBM con trazabilidad en MLflow** — 5 SP · ML · Daniel Silva

**Como** gerente de producto, **quiero** un ranking que ordene los candidatos según la probabilidad de compra, **para** que las primeras recomendaciones sean las más relevantes.

1. **Dado:** la partición leave-last-out del dataset sintético. **Cuando:** se evalúa el ranking. **Entonces:** Recall@5 ≥ 0,70 y NDCG@5 ≥ 0,48 (línea base de segmento: 0,639 y 0,435).
2. **Dado:** las recomendaciones top-5 de todos los clientes evaluados. **Cuando:** se cuentan los productos distintos. **Entonces:** la cobertura del catálogo es ≥ 80 %.
3. **Dado:** un entrenamiento del ranking. **Cuando:** termina la ejecución. **Entonces:** MLflow registra parámetros, Recall@5, NDCG@5, cobertura y el modelo en formato de texto de LightGBM (sin pickle), y al recargarlo las predicciones son idénticas (tests/test_ranker.py).

**Estimación 5 SP:** Historia más incierta del sprint: ingeniería de variables, ajuste de hiperparámetros, registro en MLflow y una meta de métricas que puede no alcanzarse al primer intento. Es la mayor; no llega a 8 porque el pipeline de evaluación ya existe (HU-02).

**HU-06 · API REST de recomendaciones** — 3 SP · Backend · Katherine Medina

**Como** canal digital, **quiero** un endpoint POST /recomendaciones que reciba un perfil y devuelva el top-5, **para** integrar el recomendador en la app y en la consola de asesores.

1. **Dado:** una petición que incluye un campo no definido como «email». **Cuando:** se envía a POST /recomendaciones. **Entonces:** la API responde HTTP 422 y no procesa el perfil.
2. **Dado:** un perfil válido. **Cuando:** se envía a POST /recomendaciones. **Entonces:** la respuesta trae cinco elementos con producto, puntaje y razón legible.
3. **Dado:** el servicio en ejecución en CPU. **Cuando:** se consulta GET /salud y se envían 1.000 peticiones a POST /recomendaciones. **Entonces:** /salud responde HTTP 200 y la latencia p95 es menor de 200 ms (tests/test_api.py con TestClient).

**Estimación 3 SP:** Patrón conocido (FastAPI + Pydantic) que arranca con un recomendador simulado; el esfuerzo está en la validación estricta y las pruebas de contrato. Igual a HU-03.

**HU-07 · Interfaz para asesores** — 2 SP · Frontend · Katherine Medina

**Como** asesor, **quiero** una pantalla donde ingrese el perfil y vea las recomendaciones con su explicación, **para** usar la sugerencia en la conversación con el cliente.

1. **Dado:** el código de la vista. **Cuando:** se revisa en el PR. **Entonces:** toda recomendación se obtiene llamando a la API, sin lógica de negocio en la UI.
2. **Dado:** un perfil ingresado en el formulario. **Cuando:** el asesor pulsa «Recomendar». **Entonces:** ve el top-5 con la razón de cada producto y el aviso de que la decisión final es suya.
3. **Dado:** la API simulada. **Cuando:** se ejecuta tests/test_ui_smoke.py. **Entonces:** la aplicación carga y muestra cinco resultados sin errores.

**Estimación 2 SP:** Vista delgada en Streamlit que solo consume la API ya definida; poca lógica y bajo riesgo. Igual a la referencia de 2 SP.

**HU-08 · Docker CPU e integración continua** — 2 SP · Testing · Daniel Silva

**Como** equipo de operaciones, **quiero** una imagen Docker reproducible y un pipeline de CI, **para** ejecutar el sistema igual en cualquier máquina y en la nube de la compañía.

1. **Dado:** un equipo limpio sin GPU. **Cuando:** se ejecuta docker build y luego docker run sin red. **Entonces:** la API arranca con PyTorch CPU y los pesos precargados.
2. **Dado:** la imagen construida. **Cuando:** se inspecciona su tamaño y sus capas. **Entonces:** pesa ≤ 2,5 GB y no contiene archivos de data/ (los datos reales se montan como volumen).
3. **Dado:** un PR hacia develop. **Cuando:** corre GitHub Actions. **Entonces:** se ejecutan ruff, pytest con cobertura ≥ 85 % y el build de la imagen, y el PR no se fusiona si alguno falla.

**Estimación 2 SP:** Dockerfile multi-stage y ajuste del workflow existente; el único riesgo conocido es libgomp para LightGBM. Igual a la referencia de 2 SP.

**HU-09 · Auditoría de sesgo por segmento** — 1 SP · Testing · Katherine Medina

**Como** responsable de cumplimiento, **quiero** conocer el desempeño del recomendador por estrato, ciudad y género, **para** detectar tratos desiguales antes de usar datos reales.

1. **Dado:** el ranking entrenado. **Cuando:** se ejecuta el script de auditoría. **Entonces:** se genera un reporte con Recall@5 por estrato, ciudad y género.
2. **Dado:** el reporte de auditoría. **Cuando:** la brecha máxima entre grupos supera 10 puntos porcentuales. **Entonces:** se abre automáticamente un issue de mitigación etiquetado Testing.
3. **Dado:** datos controlados con brecha conocida. **Cuando:** se ejecuta tests/test_auditoria.py. **Entonces:** la brecha calculada coincide con la esperada.

**Estimación 1 SP:** Reutiliza las métricas de HU-02 agrupadas por columna; un script y una prueba. Es la historia más pequeña.

**HU-10 · Integración, pruebas de humo y entrega** — 1 SP · Testing · Equipo (Daniel y Katherine)

**Como** equipo del proyecto, **quiero** integrar todos los módulos y ensayar la demo, **para** entregar el M3 funcionando en vivo.

1. **Dado:** la imagen final. **Cuando:** se ejecuta docker run. **Entonces:** API y UI quedan arriba y el flujo perfil → top-5 con razón funciona en vivo.
2. **Dado:** develop con todas las historias fusionadas. **Cuando:** se abre el PR develop → main. **Entonces:** el Code Reviewer lo aprueba con checks en verde y se etiqueta la release.
3. **Dado:** una persona ajena al proyecto. **Cuando:** sigue el README. **Entonces:** instala con uv y reproduce los resultados sin ayuda.

**Estimación 1 SP:** Ensamble final de piezas ya probadas; un día del equipo completo para el ensayo y la release.

# 6. Anexos y enlaces

## 6.1 Repositorio y artefactos

| Artefacto | Ubicación |
|---|---|
| Repositorio (src layout, uv, pytest, ruff) | github.com/daalrasi/gpia-recomendador-bolivar · privado por contener el trabajo con la compañía; se muestra en vivo y se da acceso al docente si lo solicita. La evidencia necesaria está en este anexo (6.2) |
| Tablero Kanban | github.com/users/daalrasi/projects/1 (vinculado al repositorio; captura en la Figura 4) |
| Model Card del modelo base | huggingface.co/intfloat/multilingual-e5-small |
| Generador y contrato de datos | `src/recomendador/data/sintetico.py`, `contrato.py` |
| Líneas base y métricas | `src/recomendador/baseline/`, `evaluacion/`; `uv run recomendador baseline` |
| Backlog (fuente del Cap. 5 y del tablero) | `docs/backlog.json`, `scripts/crear_kanban.py` |

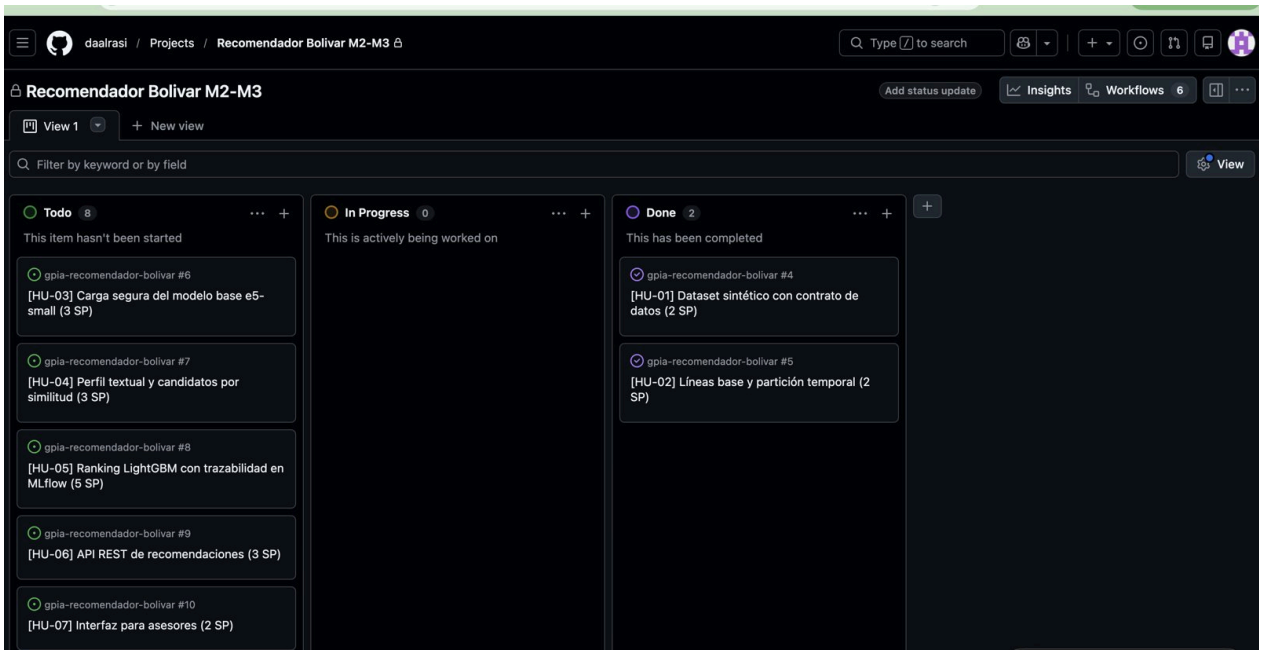



Figura 4. Tablero Kanban en GitHub Projects sincronizado con el Capítulo 5: HU-01 y HU-02 en Done y las historias del sprint M3 en Todo, con sus Story Points en el título.

## 6.2 Evidencia del repositorio

```
.github/PULL_REQUEST_TEMPLATE.md
.github/workflows/ci.yml
docs/backlog.json
pyproject.toml
scripts/check_commit_msg.py
scripts/crear_kanban.py
scripts/eda.py
src/recomendador/__init__.py
src/recomendador/baseline/__init__.py
src/recomendador/baseline/popularidad.py
src/recomendador/cli.py
src/recomendador/config.py
src/recomendador/data/__init__.py
src/recomendador/data/catalogo.py
src/recomendador/data/contrato.py
src/recomendador/data/sintetico.py
src/recomendador/evaluacion/__init__.py
src/recomendador/evaluacion/metricas.py
src/recomendador/evaluacion/particion.py
src/recomendador/py.typed
tests/test_contrato.py
tests/test_evaluacion.py
tests/test_sintetico.py
uv.lock
```

Resultado de `uv run pytest` al generar este documento: 48 pruebas en verde (patrón AAA, datos sintéticos, sin red) y cobertura de 98 %. Herramientas: Python 3.13 con uv (pyproject.toml y uv.lock), ruff y pre-commit, CI en GitHub Actions, ramas Gitflow (main, develop, feature/*) y PR revisados por la Code Reviewer.

## 6.3 Diccionario de datos

| Tabla | Columnas |
|---|---|
| clientes | cliente_id, edad, genero, ciudad, estrato, ocupacion, ingreso_smmlv, estado_civil, num_hijos, vehiculo, vivienda, arrienda_inmueble, tiene_mascota, tiene_credito, cliente_banco_aliado, canal_preferido, fecha_vinculacion |
| productos | product_id, nombre, linea, descripcion, prima_base_cop, edad_min, edad_max |
| polizas | poliza_id, cliente_id, product_id, fecha_inicio, estado, fecha_fin, canal_venta, prima_mensual_cop |
| siniestros | siniestro_id, poliza_id, fecha, monto_cop, estado |

## 6.4 Licencias

Código del proyecto: MIT. multilingual-e5-small: MIT. LightGBM: MIT. FastAPI: MIT. Streamlit: Apache 2.0. MLflow: Apache 2.0. Datos sintéticos: generados por el equipo, sin restricción.

## 6.5 Referencias (IEEE)


1. L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder y F. Wei, “Multilingual E5 Text Embeddings: A Technical Report,” arXiv:2402.05672, 2024.
2. N. Reimers e I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” en *Proc. EMNLP-IJCNLP*, 2019.
3. G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” en *Advances in NeurIPS*, vol. 30, 2017.
4. M. Mitchell et al., “Model Cards for Model Reporting,” en *Proc. ACM FAT**, 2019.
5. Parlamento Europeo y Consejo, Reglamento (UE) 2024/1689 (Ley de Inteligencia Artificial), 2024.
6. ISO/IEC 42001:2023, *Information technology — Artificial intelligence — Management system*, 2023.
7. Congreso de Colombia, Ley Estatutaria 1581 de 2012, protección de datos personales.
8. C. J. C. Burges, “From RankNet to LambdaRank to LambdaMART: An Overview,” Microsoft Research, MSR-TR-2010-82, 2010.
9. Prior Labs, “TabPFN models and licenses,” docs.priorlabs.ai, consultado el 28 de octubre de 2026.
10. C. Shearer, “The CRISP-DM Model: The New Blueprint for Data Mining,” *J. Data Warehousing*, vol. 5, n.º 4, 2000.
11. Google Cloud, “Cloud Run pricing,” cloud.google.com/run/pricing, consultado el 29 de octubre de 2026.
12. Anthropic, “Claude API pricing,” docs.claude.com, consultado el 29 de octubre de 2026.